
MakeSense Development Kit

Workshop Eurohaptics 2026

June 26, 2026

This document is intended to help you set up the haptic hardware kit used in the Applied Experimental Methods course at TU Delft. The kit is based on the BLACK PILL STM32F401X microcontroller and can be programmed using the Arduino IDE. In this document, you will find example programs to assist you in creating your own experimental setup. For any questions not handled in this document, please reach out to the course assistants.

CONTENTS

1	Introduction	3
2	Black Pill STM32F411	4
2.1	Black Pill STM32F401x in Arduino IDE	4
2.2	Testing the Board with a Simple Program	6
3	Control Peripherals	7
3.1	PIXI Click Interface Board	7
3.1.1	Connection	7
3.1.2	Libraries and software support	8
3.1.3	PIXI Click input and output examples	8
3.1.4	Practical notes	12
3.2	Audio Amplifier Click Board	12
3.2.1	Connections	12
3.2.2	Practical notes	13

3.3	L298N Motor Driver Board	13
3.3.1	Connections	13
3.3.2	Practical notes	14
4	Sensing Peripherals	14
4.1	10-DOF IMU Module	14
4.1.1	Connections	15
4.1.2	Libraries	15
4.1.3	Examples	15
4.2	Load Cell with AD620 Signal Amplifier Module	21
4.3	FSR	21
4.4	Thermistor	22
5	Actuators	22
5.1	Voice Coil Haptic Actuator	22
5.1.1	Connections	23
5.1.2	Practical notes	23
5.2	Wide Band Linear Resonant Actuator	23
5.2.1	Connections	23
5.2.2	Practical notes	24
5.3	Coin-type Linear Resonant Actuator	24
5.3.1	Connections	24
5.3.2	Practical notes	25
5.4	Narrow Band Linear Resonant Actuator	25
5.4.1	Connection	25
5.4.2	Practical notes	25
5.5	Thermoelectric Peltier Module	26
5.5.1	Connections	26
5.5.2	Practical notes	27

1 INTRODUCTION

This document provides a practical guide for setting up and using the haptic development kit employed in the *Applied Experimental Methods* course at TU Delft. The kit is centered around the Black Pill STM32F411 microcontroller, which serves as the main controller for reading sensors, interfacing with peripheral boards, and driving different actuators.

The purpose of this manual is to help students become familiar with the hardware included in the kit and to provide a starting point for building their own experimental setups. The system includes several sensing and actuation components, as well as interface and conditioning boards that allow the microcontroller to communicate with mixed-signal peripherals. Through this combination of modules, the kit supports the development of experiments involving force, temperature, inertial sensing, vibrotactile feedback, and thermal actuation.

Figure 1.1 shows the complete haptic development kit used throughout this document.

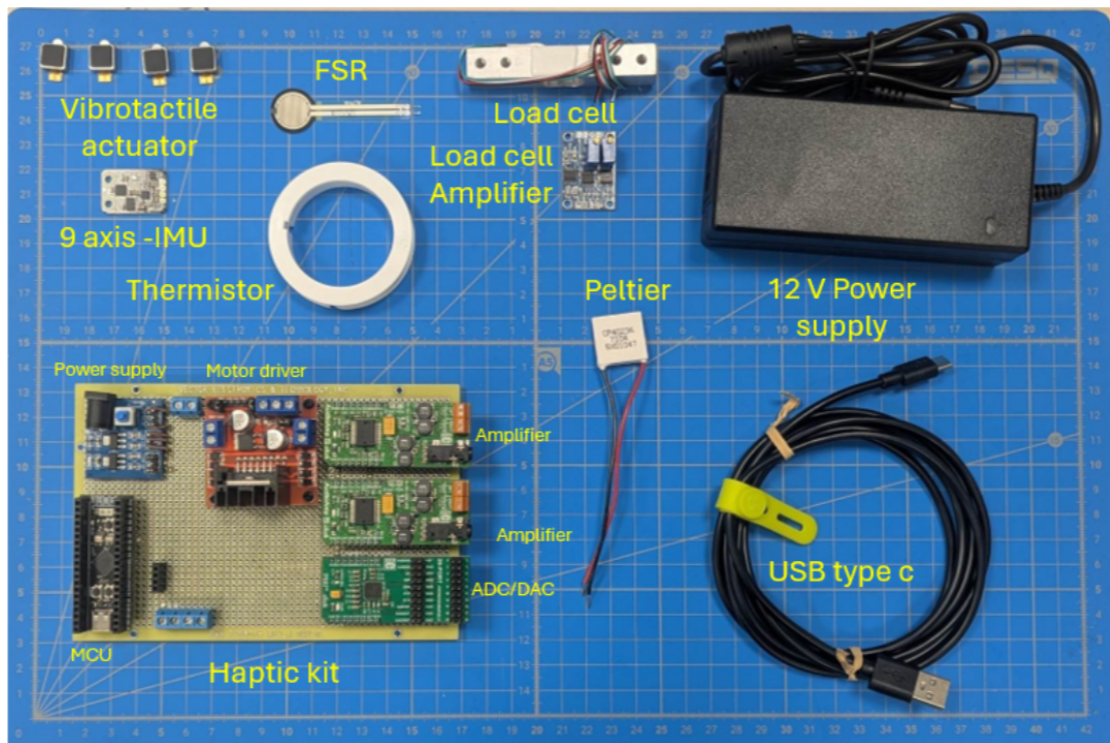


Figure 1.1: Overview of the haptic development kit used in the Applied Experimental Methods course.

A dedicated GitHub repository was created to centralize all code-related material for the AEM 2026 haptic kit. This repository includes the required software resources, relevant libraries, example codes, and implementation notes needed to set up and test the different electronic components used in the kit. It is intended to support students during the practical

work by providing a single reference point for programming the microcontroller, interfacing with sensors and actuators, and reproducing the examples described in this documentation. The repository can be accessed through the following link: [https://github.com/paulina320/AEM_2026]

2 BLACK PILL STM32F411

The Black Pill STM32F411 is the main microcontroller board used in this setup. It is based on the STM32F411CEU6 microcontroller, which features an Arm Cortex-M4 core running at up to 100 MHz, 512 KB of Flash memory, and 128 KB of SRAM. The board provides digital GPIO pins, analog inputs, PWM-capable pins, and communication interfaces such as I²C, SPI, and UART, which makes it suitable for reading sensors and controlling external modules.

In this setup, the Black Pill acts as the central controller and communicates with the Pixi Click board, which is then used as the configurable analog and digital interface for several of the connected devices. The Black Pill can be powered through USB-C or through its external power pins, while its GPIO pins are used for communication and control. The exact pins used depend on the peripherals connected in the final system, but typical examples include I²C lines for communication, SPI lines for the Pixi Click interface, and digital outputs for actuator control.

The board operates at 3.3 V logic level, so connected peripherals must be compatible with this voltage level or must be interfaced appropriately. Depending on the board version, power can be supplied through USB, 5 V pins, or regulated 3.3 V pins.

For more information check the product pages at these links:

<https://www.st.com/resource/en/datasheet/stm32f411ce.pdf> and

<https://stm32-base.org/boards/STM32F411CEU6-WeAct-Black-Pill-V2.0.html>

2.1 BLACK PILL STM32F401X IN ARDUINO IDE

In this section, you will learn how to set up the Black Pill STM32F411 board in the Arduino IDE, install the required board package, choose the correct board settings, and upload your first test program. Although this manual starts with Arduino IDE, the Black Pill can also be used with other IDEs and programming approaches if needed.

Before starting, make sure you have:

- A Black Pill STM32F411 board
- A USB to USB-C cable
- Arduino IDE 2 installed on your computer

The procedure below describes how to configure the Arduino IDE and upload code to the Black Pill STM32F411 board using DFU mode.

1. Open the Arduino IDE.
2. Open the Preferences menu: Go to File > Preferences or Arduino IDE > Settings, depending on your operating system.
3. Add the STM32 Board Manager URL: In the Additional Boards Manager URLs field, add the following link: https://github.com/stm32duino/BoardManagerFiles/raw/main/package_stmicroelectronics_index.json
4. Install the STM32 board package in Arduino IDE: Go to Tools > Board > Boards Manager, search for STM32 MCU based boards, and install the latest version published by STMicroelectronics.
5. Install STM32CubeProgrammer: Download and install STM32CubeProgrammer from <https://www.st.com/en/development-tools/stm32cubeprog.html>. This software is required for uploading code to the STM32 board when using STM32CubeProgrammer (DFU) as the upload method.
6. Select the correct board settings in Arduino IDE: After installation, go to Tools and configure the board as follows:
 - **Board:** Generic STM32F4 series
 - **Board part number:** BlackPill F411CE
 - **Upload method:** STM32CubeProgrammer (DFU)
 - **USB support:** CDC (generic 'Serial' supersede U(S)ART)
 - **USB speed:** Low/Full Speed
7. Connect the board to the computer using a USB-C to USB cable.
8. Put the board into DFU mode: To upload code through DFU, the board must first be placed into DFU mode. The recommended method is to disconnect the board from the computer, press and hold the BOOT button, reconnect the board through USB while keeping the button pressed, and then release the BOOT button once the board is connected.

An alternative method is, while the microcontroller is powered on through the USB C cable, to press and hold the BOOT button, briefly press and release the RESET button, and then release the BOOT button. However, this method is less reliable and may result in inconsistent device detection by the computer.

9. Open the Tools menu in the Arduino IDE and check whether the board is detected correctly. If a port associated with the board appears under Port, select it before proceeding. If no port appears at this stage, continue with the upload process; after the

code is uploaded successfully, the board will typically appear as a serial device.

10. Upload the code to the board.

2.2 TESTING THE BOARD WITH A SIMPLE PROGRAM

The first program uploaded to the board should be as simple as possible. Its purpose is to confirm that the board is correctly detected, the code compiles without errors, the upload process works properly, and the microcontroller is running as expected.

One of the easiest programs for this initial verification is a basic LED blink example, shown below. This code is also available in the GitHub repository under the example name Blink.

```
1  #define LED_PIN PC13
2
3  void setup() {
4      pinMode(LED_PIN, OUTPUT);
5      Serial.begin(115200);
6      delay(1000);
7
8      Serial.println("STM32 Black Pill is starting...");
9  }
10
11 void loop() {
12     digitalWrite(LED_PIN, LOW);    // LED on
13     Serial.println("LED ON");
14     delay(500);
15
16     digitalWrite(LED_PIN, HIGH);   // LED off
17     Serial.println("LED OFF");
18     delay(500);
19 }
```

This is one of the simplest and most commonly used test programs for checking whether the board is working correctly. However, other basic examples can also be used for this purpose, such as simple serial communication tests or programs that read a digital input. Students are encouraged to explore additional beginner-level examples in Arduino IDE once the board has been successfully configured.

1. Upload the example program shown above, or another simple test program of your choice.
2. For this example, the onboard LED should blink continuously. On Black Pill boards, the onboard LED is typically connected to PC13. However, when supported by the selected board definition, it is preferable to use LED_BUILTIN.
3. Before each upload, the board must be placed into DFU mode. To do this, press and

hold the BOOT button, then connect the board to the computer via USB while keeping the button pressed. Once connected, you can release the BOOT button.

4. If the selected program requires external libraries, install them first in Arduino IDE by going to `Sketch > Include Library > Manage Libraries`. Search for the required library, install it, and include it in the code using the corresponding `#include` statement.
5. After the upload is complete, verify that the program behaves as expected. For the LED blink example, confirm that the onboard LED starts blinking. If the program also sends messages through the serial port, open the Serial Monitor and check that they are displayed correctly.
6. If the expected behavior is observed, the board has been successfully configured and is ready for further development.

3 CONTROL PERIPHERALS

3.1 PIXI CLICK INTERFACE BOARD

The PIXI Click board is used in this kit as an interface between the STM32 Black Pill microcontroller and mixed-signal peripherals. It is based on the MAX11300 mixed-signal data converter, which provides configurable analog and digital input/output functions. This makes it suitable for acquiring signals from external circuits and for supporting communication between the microcontroller and additional sensing or conditioning stages.

In this setup, the PIXI Click board communicates with the STM32 Black Pill through the SPI interface. The SPI lines are used for data exchange between the microcontroller and the MAX11300, while additional control pins such as chip select, reset, interrupt, and conversion trigger are used for device control and synchronization. Through this interface, the Black Pill can configure the ports of the PIXI Click board and read or write signals depending on the connected peripheral.

Before integrating the PIXI Click board into a complete measurement system, it is recommended to first establish and verify its communication with the STM32 Black Pill, in order to ensure that the SPI interface and the control pins are working correctly.

For more information check the product page at this link:

<https://www.mikroe.com/pixi-click>

3.1.1 CONNECTION

The PIXI Click board is connected to the STM32 Black Pill through the SPI interface, together with the required power and control signals.

The required signal groups are:

- **SPI communication lines:** MOSI, MISO, SCLK, and Chip Select
- **Control lines:** CNVT and INT
- **Power lines:** 3.3V, 5V, and GND

Important: The PIXI Click board communicates through SPI, not through I²C. Therefore, the SDA and SCL pins of the STM32 Black Pill are not used for this interface.

3.1.2 LIBRARIES AND SOFTWARE SUPPORT

The PIXI Click board is based on the MAX11300 mixed-signal converter and communicates through SPI. This board requires software configuration of its internal ports before it can be used, since each port can be programmed individually for functions such as analog input, analog output, digital input, digital output, or analog switching.

Download the PIXI/MAX11300 support files from the GitHub repository introduced at the beginning of this document. These files are not installed as a formal Arduino library yet; instead, they are included locally in each project that uses the PIXI Click board. The required files are “MAX11300.h”, “MAX11300.cpp”, and “MAX11300registers.h”.

In the example folders provided in the repository, these files are already included, so the examples can be compiled directly. However, when creating a new sketch or project that uses the PIXI Click, the three files must be copied into the same folder as the main code file. This ensures that the code can access the MAX11300 definitions, register information, and helper functions needed to communicate with the board.

The provided Arduino support files simplify the use of the PIXI Click by hiding most of the low-level communication details. They provide easy-to-use functions for configuring the PIXI pins and for reading or writing data, making it easier to integrate analog inputs, analog outputs, and digital I/O into new projects.

3.1.3 PIXI CLICK INPUT AND OUTPUT EXAMPLES

The GitHub repository includes different PIXI Click examples for reading sensor signals and generating output signals for actuation. These examples are useful for verifying the communication between the PIXI Click and the STM32 Black Pill, and for understanding how the MAX11300 ports can be configured either as analog inputs or analog outputs.

READING ANALOG DATA FROM SENSORS

To verify that the communication between the PIXI Click and the STM32 microcontroller is working correctly, use the example code below to configure one PIXI port as an analog input and read its value. This code is also available in the GitHub repository under `Read_Analog_Pin`.

```
1
2 #include "MAX11300.h"
3
4 #define CNVT_PIN PBO
5 #define CS_PIN PA4
6
7 MAX11300 pixi(&SPI, CNVT_PIN, CS_PIN);
8
9 const uint8_t IN_PIN = 6;
10
11 void setup() {
12     Serial.begin(115200);
13     delay(1000);
14
15     pixi.begin();
16
17     pixi.setPinMode(IN_PIN, analogIn);
18     pixi.setPinADCrang(IN_PIN, ADCZeroTo2_5);
19     pixi.setPinADCref(IN_PIN, ADCInternal);
20     pixi.setADCmode(ContinuousSweep);
21
22     Serial.println("Setup done");
23 }
24
25 void loop() {
26     uint16_t val = pixi.readAnalogPin(IN_PIN);
27     Serial.print("Read Analog: ");
28     Serial.println(val);
29 }
```

In this example, PIXI port 6 is configured as an analog input using the internal ADC reference and a 0–2.5 V input range. The code continuously reads the selected port and prints the ADC value to the Serial Monitor. If the selected pin is connected to GND, the measured value should be close to 0. If it is connected to VCC within the configured input range, the reading should be close to the maximum value. If the pin is left unconnected, the readings may fluctuate due to electrical noise.

For sensor-based measurements, the repository also includes a more complete reading example under `Read_Data`. This example is intended for reading analog sensor signals connected to the PIXI Click, such as force sensors or other analog-output sensors. When the key button is pressed, the Arduino sketch `Read_Data.ino` reads two analog input ports and sends the measured values through the serial COM port. The corresponding Python script, `Read_Data.py`, receives the serial data and stores it in a CSV file. The CSV file is saved in

the same directory from which the Python script is executed. This is useful when sensor data needs to be recorded for later analysis, plotting, or comparison between experiments.

WRITING ANALOG SIGNALS FOR ACTUATOR CONTROL

The PIXI Click can also be used to generate analog output signals. The following example configures one PIXI port as an analog output and generates a sinusoidal signal when the button is pressed. This type of output can be used as a control signal for external actuator-driving electronics, such as the AudioAmp 2 Click used to drive tactile actuators.

```
1  #include <SPI.h>
2  #include "MAX11300.h"
3
4  #define M_PI 3.1415926535897932384626433832795f
5
6  #define CNVT_PIN PBO
7  #define CS_PIN PA4
8  #define BTN_PIN PA0
9
10 #define OUT_PIN0 0
11
12 MAX11300 pixi(&SPI, CNVT_PIN, CS_PIN);
13
14
15 void setup() {
16
17     Serial.begin(115200);
18
19     pinMode(BTN_PIN, INPUT_PULLUP);
20
21     // Initialize the MAX11300
22     pixi.begin();
23
24     delay(1); // Allow chip to stabilize
25
26     pixi.setDACref(DACInternal);
27     pixi.setPinMode(OUT_PIN0, analogOut); //FUNCID
28
29     delay(200); // from datasheet
30
31     pixi.setPinDACrange(OUT_PIN0, DACZeroTo10); //FUNCPRM
32
33     delayMicroseconds(200); // from datasheet
34
35     pixi.writeAnalogPin(OUT_PIN0, 0);
36
37
```

```

38   Serial.println("Setup complete");
39 }
40
41 int16_t generateSample(uint16_t sample_rate_hz, float frequency_hz, float
    amplitude ){
42     static float phase_offset_rad = 0.0f;
43
44     float phase = 2.0f * M_PI * frequency_hz * 1.0f / sample_rate_hz +
    phase_offset_rad;
45     float sample = (amplitude * sinf(phase) + amplitude) / 2;
46
47     phase_offset_rad = fmodf(phase, 2.0f * M_PI);
48
49     return (int16_t)sample;
50 }
51
52
53 void loop() {
54     static unsigned long lastUpdateTime = 0;
55     const unsigned long updateInterval = 20; // 20 microseconds interval
56
57     unsigned long currentTime = micros();
58
59     // Wait until it's time for the next update
60     if (currentTime - lastUpdateTime >= updateInterval) {
61         lastUpdateTime = currentTime;
62
63         int buttonState = digitalRead(BTN_PIN);
64
65         if(buttonState == LOW) {
66             int16_t val = generateSample(50000, 150, 100);
67             pixi.writeAnalogPin(OUT_PIN0, val + 150);
68         }
69         else {
70             pixi.writeAnalogPin(OUT_PIN0, 0);
71         }
72     }
73 }

```

In this example, PIXI port 0 is configured as an analog output with a 0–10 V DAC range. The function `generateSample()` computes a sine-wave sample based on the selected sample rate, frequency, and amplitude. In the main loop, the code checks the state of the button connected to PA0. When the button is pressed, the generated signal is written to the PIXI output port. When the button is released, the output is set back to 0.

This output signal should not be used to power an actuator directly. Instead, it is used as a signal input to an external driver or amplifier. For example, the PIXI analog output can be connected to the input of the AudioAmp 2 Click, which then provides the required power to

drive a tactile actuator such as a voice-coil or vibration actuator. This separation is important because the PIXI Click provides configurable signal-level analog outputs, while the actuator driver or amplifier provides the higher current required for actuation.

3.1.4 PRACTICAL NOTES

The MAX11300 integrates a single ADC that is shared across all configured analog input channels through time multiplexing. As a result, the specified maximum conversion rate (up to 400 kSPS) represents the total sampling rate, which is distributed among all active channels. Therefore, the effective sampling rate per channel decreases as the number of active channels increases.

In addition, the ADC supports sample averaging, where multiple consecutive measurements are combined to produce a single output value.

3.2 AUDIO AMPLIFIER CLICK BOARD

The AudioAmp 2 Click is an audio amplifier expansion board from MikroElektronika based on the TDA7491, a dual BTL class-D audio amplifier integrated circuit. It is intended for audio amplification applications and can deliver up to 20 W to an 8 Ω load under appropriate operating conditions. The board includes a 3.5 mm stereo line input, left and right speaker output terminals, selectable gain settings, mute and standby control, and a diagnostic output for fault monitoring.

For more information check the product pages at these links:

<https://www.mikroe.com/audioamp-2-click> and

<https://download.mikroe.com/documents/datasheets/TDA7491HV.pdf>

3.2.1 CONNECTIONS

The AudioAmp 2 Click can be controlled through the Pixi Click board, which is based on the MAX11300 programmable mixed-signal I/O integrated circuit. In this configuration, the Pixi Click is used both for digital control and for generating the analog waveform to be amplified using one of its DAC output.

- **Power lines:** External supply 5V and GND
- **Control lines:** GN0, GN1 (gain selection), STB (standby), and MUTE
- **Analog signal line:** Pixi Click analog output (DAC) → AudioAmp 2 Click line input (via AUX cable)
- **Status line (optional):** DIA (diagnostic output) to Pixi Click or Black Pill input

- **Reference Pin:** AVDDIO → 5V and AVSSIO → GND
- **Output lines:** OUT+ and OUT– for connection with the actuator

3.2.2 PRACTICAL NOTES

1. The AudioAmp 2 Click is designed as an audio amplifier, so it is most suitable for loads that require an AC excitation signal.
2. The output is a BTL output stage, so the load should be connected directly across the output terminals and not referenced to ground.
3. The gain configuration should be selected carefully to avoid excessive output amplitude and possible overdrive of the connected actuator or speaker.
4. If this board is used with a haptic actuator, the actuator electrical limits must be checked carefully before operation.
5. The Pixi Click in this haptic kit is configured to an 20 dB gain by pulling GN0 and GN1 low.

3.3 L298N MOTOR DRIVER BOARD

The L298N Motor Driver Board is a driver module based on the ST L298 dual full-bridge driver integrated circuit. It can be used to control two DC motors or one stepper motor, and the product page specifies a current capability of 2 A per channel. The underlying L298 device is a dual full-bridge driver intended for inductive loads such as relays, solenoids, DC motors, and stepper motors.

For more information check the product page at these links: <https://www.benselectronics.nl/l298n-motor-driver-board.html>

3.3.1 CONNECTIONS

The L298N Motor Driver Board can be controlled through the Pixi Click board, which is based on the MAX11300 programmable mixed-signal I/O integrated circuit. In this configuration, the Pixi Click provides the digital control signals for the L298N, while the external power supply provides the load current. Since the L298N is a full-bridge driver, one channel typically uses two logic inputs to define the output polarity and one enable input to activate the channel or apply PWM control.

- **Power lines:** External motor supply (Vs), logic supply (5 V), and GND
- **Control lines:** IN1, IN2, IN3, and IN4 (direction control)

- **Enable/PWM lines:** ENA and ENB (channel enable or PWM speed control)
- **Load connection:** OUT1, OUT2 (Channel A) and OUT3, OUT4 (Channel B)

The four control signals connected to inputs in the Motor Driver define the polarity applied to the load. If the **ENA** input is also driven from the Pixi Click, the channel can be enabled or disabled by software. If the board is used with the onboard enable jumper left in place, only the inputs need to be controlled. This makes the Pixi Click suitable as a flexible logic interface between the Black Pill and the L298N board.

3.3.2 PRACTICAL NOTES

1. The product page states that the module can be used for **2 x DC motors or 1 x stepper motor** and specifies **2 A per channel**.
2. The L298 is a **dual full-bridge driver**, so it is suitable for bidirectional control of DC loads.
3. The load current must be supplied from an external power source; the Black Pill and the Pixi Click provide only logic-level control signals. This is an engineering consequence of the L298 being a separate power driver stage.
4. The L298 introduces a noticeable voltage drop across the output stage, so the voltage seen by the load will be lower than the applied supply voltage. This follows from the bipolar full-bridge architecture described in the L298 datasheet.

4 SENSING PERIPHERALS

4.1 10-DOF IMU MODULE

The IMU module used in this kit is the **DFRobot Fermion 10 DOF IMU Sensor V2 (SEN0140)**. Unlike single-chip IMUs, this module combines four different sensors on the same board and communicates through the I²C interface. The integrated sensors are:

- **ADXL345:** 3-axis accelerometer
- **ITG3200 / ITG3205:** 3-axis gyroscope
- **VCM5883L:** 3-axis magnetometer
- **BMP280:** barometric pressure and temperature sensor

Because the module contains several independent devices, the recommended workflow is to test each sensor separately before combining all measurements in one program.

For more information check the product pages at these links: <https://www.dfrobot.com/product-818.html>

4.1.1 CONNECTIONS

Connect the IMU module to the STM32 Black Pill board using the I²C interface:

- **VCC → 3.3V**
- **GND → GND**
- **SDA → I²C SDA pin**
- **SCL → I²C SCL pin**

Important: Even though the module includes an onboard regulator and accepts a wider power input range, the STM32 Black Pill operates with 3.3V logic, so using 3.3V is the safest option for I²C communication.

4.1.2 LIBRARIES

For this module, use the libraries referenced in the DFRobot documentation.

- **For orientation:**
 - DFRobot_ADXL345
 - DFRobot_ITG3200
 - DFRobot_QMC5883
- **For barometer and temperature measurements:**
 - DFRobot_BMP280

These libraries should be downloaded and installed in the Arduino IDE before uploading the example programs.

4.1.3 EXAMPLES

BAROMETER AND TEMPERATURE TEST

The following example verifies the communication with the BMP280 sensor integrated in the IMU module. The code reads the sensor data and prints the measured temperature, atmospheric pressure, and estimated altitude values to the Serial Monitor. This example is also

available in the GitHub repository under Barometer_temperature.

```
1 #include <Wire.h>
2 #include "DFRobot_BMP280.h"
3
4 typedef DFRobot_BMP280_IIC BMP;
5 BMP bmp(&Wire, BMP::eSdoLow);
6
7 #define SEA_LEVEL_PRESSURE 1013.25f
8
9 void setup() {
10     Serial.begin(115200);
11     delay(1000);
12     Serial.println("BMP280 demo on STM32 Black Pill");
13
14     Wire.begin(PB7, PB6);    // SDA, SCL
15
16     while (bmp.begin() != BMP::eStatusOK) {
17         Serial.println("BMP280 init failed, check wiring.");
18         delay(2000);
19     }
20
21     Serial.println("BMP280 initialized correctly.");
22 }
23
24 void loop() {
25     float temperature = bmp.getTemperature();
26     uint32_t pressure = bmp.getPressure();
27     float altitude = bmp.calAltitude(SEA_LEVEL_PRESSURE, pressure);
28
29     Serial.print("Temperature = ");
30     Serial.print(temperature);
31     Serial.println(" *C");
32
33     Serial.print("Pressure = ");
34     Serial.print(pressure);
35     Serial.println(" Pa");
36
37     Serial.print("Altitude = ");
38     Serial.print(altitude);
39     Serial.println(" m");
40
41     Serial.println();
42     delay(2000);
43 }
44 }
```

After uploading the code, open the Serial Monitor to verify that the temperature, pressure, and altitude values are printed correctly.

COMBINED IMU EXAMPLE

This example demonstrates how to use the STM32 Black Pill as an I²C reader for the DFRobot SEN0140 10-DOF IMU module, using PB7 as SDA and PB6 as SCL. The program initializes the different sensors integrated in the module: the ADXL345 accelerometer, the ITG3200 gyroscope, the QMC/VCM5883-compatible magnetometer, and the BMP280 barometer. During operation, it continuously reads acceleration, angular velocity, magnetic field, temperature, pressure, and estimated altitude values, and prints them to the Serial Monitor. This allows the user to verify that all sensors on the SEN0140 board are communicating correctly and to observe live motion and environmental data from the module. This example is also available in the GitHub repository under DFRobot_IMU_BlackPill.

```
1  #include <Wire.h>
2  #include "DFRobot_ADXL345.h"
3  #include "DFRobot_BMP280.h"
4  #include "DFRobot_QMC5883.h"
5
6  DFRobot_ADXL345_I2C accel(&Wire, ADXL345_ADDR_ALT_LOW);
7  DFRobot_QMC5883 compass(&Wire, VCM5883L_ADDRESS);
8  typedef DFRobot_BMP280_IIC BMP;
9  BMP bmpLow(&Wire, BMP::eSdoLow);
10 BMP bmpHigh(&Wire, BMP::eSdoHigh);
11 BMP *bmp = nullptr;
12
13 #define SEA_LEVEL_PRESSURE 1013.25f
14
15 constexpr uint8_t ITG3200_ADDR = 0x68;
16 constexpr uint8_t ITG3200_REG_PWR_MGM = 0x3E;
17 constexpr uint8_t ITG3200_REG_SMPLRT_DIV = 0x15;
18 constexpr uint8_t ITG3200_REG_DLPF_FS = 0x16;
19 constexpr uint8_t ITG3200_REG_GYRO_XOUT = 0x1D;
20 constexpr float ITG3200_SCALE_DPS = 1.0f / 14.375f;
21
22 bool writeRegister(uint8_t address, uint8_t reg, uint8_t value) {
23     Wire.beginTransmission(address);
24     Wire.write(reg);
25     Wire.write(value);
26     return Wire.endTransmission() == 0;
27 }
28
29 bool readRegisters(uint8_t address, uint8_t reg, uint8_t *buffer, uint8_t
    length) {
30     Wire.beginTransmission(address);
31     Wire.write(reg);
32     if (Wire.endTransmission(false) != 0) {
33         return false;
34     }
35 }
```

```

35
36     uint8_t count = Wire.requestFrom(address, length);
37     if (count != length) {
38         return false;
39     }
40
41     for (uint8_t i = 0; i < length; i++) {
42         buffer[i] = Wire.read();
43     }
44     return true;
45 }
46
47 int16_t makeInt16(uint8_t msb, uint8_t lsb) {
48     return (int16_t)((msb << 8) | lsb);
49 }
50
51 bool initGyro() {
52     return writeRegister(ITG3200_ADDR, ITG3200_REG_PWR_MGM, 0x00) &&
53           writeRegister(ITG3200_ADDR, ITG3200_REG_SMPLRT_DIV, 0x07) &&
54           writeRegister(ITG3200_ADDR, ITG3200_REG_DLPF_FS, 0x1B);
55 }
56
57 bool readGyro(float &gx, float &gy, float &gz) {
58     uint8_t raw[6];
59     if (!readRegisters(ITG3200_ADDR, ITG3200_REG_GYRO_XOUT, raw, 6)) {
60         return false;
61     }
62
63     gx = makeInt16(raw[0], raw[1]) * ITG3200_SCALE_DPS;
64     gy = makeInt16(raw[2], raw[3]) * ITG3200_SCALE_DPS;
65     gz = makeInt16(raw[4], raw[5]) * ITG3200_SCALE_DPS;
66     return true;
67 }
68
69 void setup() {
70     Serial.begin(115200);
71     delay(1000);
72
73     Wire.begin(PB7, PB6);
74     Wire.setClock(100000);
75
76     Serial.println("DFRobot SEN0140 accel + gyro test on STM32 Black Pill");
77     Serial.println("Wiring:");
78     Serial.println("  VIN -> 3V3");
79     Serial.println("  GND -> GND");
80     Serial.println("  SDA -> PB7");
81     Serial.println("  SCL -> PB6");
82
83     if (!accel.begin()) {

```

```

84     Serial.println("ADXL345 init failed");
85     while (1) {
86     }
87 }
88 accel.powerOn();
89 Serial.println("ADXL345 initialized correctly.");
90
91 Serial.println("Starting ITG3200...");
92 if (!initGyro()) {
93     Serial.println("ITG3200 init failed");
94     while (1) {
95     }
96 }
97 Serial.println("ITG3200 initialized correctly.");
98
99 Serial.println("Starting magnetometer...");
100 if (!compass.begin()) {
101     Serial.println("Magnetometer init failed");
102     while (1) {
103     }
104 }
105 Serial.println("Magnetometer initialized correctly.");
106
107 Serial.println("Starting BMP280...");
108 if (bmpLow.begin() == BMP::eStatusOK) {
109     bmp = &bmpLow;
110     Serial.println("BMP280 initialized correctly at 0x76.");
111 } else if (bmpHigh.begin() == BMP::eStatusOK) {
112     bmp = &bmpHigh;
113     Serial.println("BMP280 initialized correctly at 0x77.");
114 } else {
115     Serial.println("BMP280 init failed");
116     while (1) {
117     }
118 }
119 }
120
121 void loop() {
122     int xyz[3];
123     accel.readAccel(xyz);
124
125     float gx = 0.0f;
126     float gy = 0.0f;
127     float gz = 0.0f;
128     bool gyroOk = readGyro(gx, gy, gz);
129     sVector_t mag = compass.readRaw();
130     float temperature = bmp->getTemperature();
131     uint32_t pressure = bmp->getPressure();
132     float altitude = bmp->calAltitude(SEA_LEVEL_PRESSURE, pressure);

```

```

133
134     Serial.print("Accel X: ");
135     Serial.print(xyz[0]);
136     Serial.print("  Y: ");
137     Serial.print(xyz[1]);
138     Serial.print("  Z: ");
139     Serial.println(xyz[2]);
140
141     if (gyroOk) {
142         Serial.print("Gyro X: ");
143         Serial.print(gx);
144         Serial.print("  Y: ");
145         Serial.print(gy);
146         Serial.print("  Z: ");
147         Serial.println(gz);
148     } else {
149         Serial.println("Gyro read failed.");
150     }
151
152     Serial.print("Mag X: ");
153     Serial.print(mag.XAxis);
154     Serial.print("  Y: ");
155     Serial.print(mag.YAxis);
156     Serial.print("  Z: ");
157     Serial.println(mag.ZAxis);
158
159     Serial.print("Temperature: ");
160     Serial.print(temperature);
161     Serial.println(" *C");
162
163     Serial.print("Pressure: ");
164     Serial.print(pressure);
165     Serial.println(" Pa");
166
167     Serial.print("Altitude: ");
168     Serial.print(altitude);
169     Serial.println(" m");
170
171     Serial.println();
172     delay(500);
173 }

```

Note: This example is mainly intended as a functional test of the SEN0140 IMU module. Some values, such as temperature, pressure, altitude, and angular velocity, are printed in physical units and are therefore easier to interpret. However, the accelerometer and magnetometer outputs are printed as raw sensor values. These raw values are useful for verifying that the sensors respond to motion and orientation changes, but they should be converted and calibrated if quantitative measurements are required.

This example provides a complete functional test of the SEN0140 IMU module. If all initialization messages are printed correctly and the measured values change when the board is moved or tilted, the I²C communication and the individual sensors are working as expected. If one of the sensors fails to initialize, check the wiring, the selected I²C pins, the supply voltage, and whether the required DFRobot libraries are correctly installed.

4.2 LOAD CELL WITH AD620 SIGNAL AMPLIFIER MODULE

The load cell is a force sensor that converts mechanical deformation into a very small differential electrical signal. Since this output signal is too small to be read directly with sufficient resolution by the Pixi Click board, it is used together with the AD620 instrumentation amplifier module. The AD620 amplifies the differential voltage generated by the load cell and provides a single-ended analog output that can be measured by the Pixi Click board.

In this setup, the load cell is excited using an external 5 V supply. The red wire is connected to the positive excitation voltage, while the black wire is connected to ground. The differential output wires of the load cell, green and white, are connected to the S+ and S- inputs of the AD620 module, respectively. The AD620 module is powered from an external supply, and its output Vout is connected to the analog input pin of the Pixi Click board. The grounds of the amplifier, the load cell excitation source, and the Pixi Click board must all be common.

On the board, two potentiometers are available: one to adjust the gain and one to adjust the offset of the AD620 amplifier. During the setup phase, the gain is tuned by applying representative loads, from no load to the maximum expected load, and increasing the amplification until the signal spans most of the input range of the Pixi Click board without reaching saturation. The offset can be adjusted so that the no-load condition corresponds to a well-defined baseline voltage. Once the gain and offset are properly set, the calibration is performed by applying known weights and recording the corresponding output voltages to establish the relationship between voltage and applied force.

For more information check the product pages at these links <https://www.tinytronics.nl/en/communication-and-signals/io-converters/signal-amplifier-module-ad620> and <https://www.otronic.nl/nl/weegcel-ysz-131-sensor-tot-1-kg.html>

4.3 FSR

An FSR is a two-terminal sensor whose resistance decreases as the applied force increases. To obtain a measurable signal, the FSR is used in a voltage divider configuration with a fixed resistor, which converts the resistance variation into a voltage that can be read by an analog input. In this setup, a 10 k Ω pull-up resistor is externally soldered, so no additional resistor is required.

The FSR is connected between the analog input pin of the Pixi Click board and GND, while the pull-up resistor connects the same input node to Vcc (3.3V). As the applied force increases,

the resistance of the FSR decreases, resulting in a change in the voltage at the input, which can then be measured by the Pixi Click board.

For more information check the product pages at these links: <https://www.kiwi-electronics.com/image/catalog/pdf/FSR402.pdf>

4.4 THERMISTOR

The thermistor is a two-terminal temperature sensor whose resistance varies with temperature. In this case, a Negative Temperature Coefficient (NTC) thermistor is used, meaning that its resistance decreases as temperature increases.

Similarly to the FSR, the thermistor is usually used in a voltage divider configuration. In this setup, a 22k Ω pull-up resistor is externally soldered, forming a divider that converts resistance changes into a measurable voltage.

The thermistor is connected between the analog input pin of the Pixi Click board and GND, while the pull-up resistor connects the same node to Vcc (3.3V). As temperature increases, the resistance of the thermistor decreases, resulting in a change in the voltage at the input, which can then be measured by the Pixi Click board.

The relationship between resistance and temperature is nonlinear, as shown in the datasheet, and typically requires calibration or a lookup table for accurate temperature estimation.

For more information check the product pages at these links <https://www.semitec-global.com/uploads/2022/01/P8-F-micro-Thermistor.pdf>

5 ACTUATORS

5.1 VOICE COIL HAPTIC ACTUATOR

The Tactile Labs HapCoil-One, also referred to by the manufacturer as the Haptuator Mark II-D, is a voice coil haptic actuator intended for broadband vibrotactile feedback. Unlike a linear resonant actuator, which is typically driven near a single resonant frequency, this actuator can operate over a wider frequency range and can be driven in a manner similar to a loudspeaker. According to the manufacturer, it provides large rated bandwidth, high acceleration output, and compatibility with most audio amplifiers.

For more information check the product pages at these links <https://tactilelabs.com/product/hapcoil-one/>

5.1.1 CONNECTIONS

In each kit you will find two AUX cables to connect the Pixi Click to the audio amplifier.

- 3.5 mm AUX cable → AudioAmp 2 Click line-in jack
- Tip (left channel) → Pixi Click analog output
- Ring (right channel) → Pixi Click analog output
- Sleeve (GND) → Pixi Click GND
- AudioAmp 2 Click (+) → HapCoil-One terminal 1
- AudioAmp 2 Click (−) → HapCoil-One terminal 2

5.1.2 PRACTICAL NOTES

1. This actuator is better suited for broadband vibrotactile rendering than a narrowband LRA, since the manufacturer describes it as having large rated bandwidth.
2. The final vibration response will depend strongly on the way the actuator is mounted in the mechanical structure, so the integration should be designed carefully. This is a general engineering consideration for voice coil haptic actuators.

5.2 WIDE BAND LINEAR RESONANT ACTUATOR

The Vybronic VLV101040A is a linear resonant actuator (LRA) intended for haptic feedback applications. Unlike an eccentric rotating mass motor, an LRA generates vibration by oscillating an internal mass near its resonant frequency, which enables faster response and more controlled haptic output. This actuator has dimensions of $10 \times 10 \times 4$ mm, a Z-axis vibration direction, a rated voltage of $2.5 V_{\text{RMS}}$, an operating voltage range of 0.1 to $2.5 V_{\text{RMS}}$, a resonant frequency of 170 Hz, a rated current of 350 mA maximum, a typical current of 317 mA, and a vibration force of 2.75 G under rated conditions.

For more information check the product pages at these links: <https://www.vybronic.com/wp-content/uploads/datasheet-files/Vybronic-VLV101040A-datasheet.pdf>

5.2.1 CONNECTIONS

- 3.5 mm AUX cable → AudioAmp 2 Click line-in jack
- Tip (left channel) → Pixi Click analog output
- Ring (right channel) → Pixi Click analog output

- Sleeve (GND) → Pixi Click GND
- AudioAmp 2 Click (+) → LRA terminal 1 (red)
- AudioAmp 2 Click (−) → LRA terminal 2 (black)

5.2.2 PRACTICAL NOTES

1. Best performance is achieved when the actuator is driven close to its resonant frequency of 170 Hz.
2. Because this actuator has a wide bandwidth, haptic drivers using auto-resonance detection may require manual configuration.
3. The mounting condition affects the vibration response, so the actuator should be fixed carefully in the final mechanical structure. This part is designed for pressure contacts such as pogo pins or spring contacts.

5.3 COIN-TYPE LINEAR RESONANT ACTUATOR

The Vybronic VG0832013D is a coin-type linear resonant actuator (LRA) intended for haptic feedback applications. Unlike an eccentric rotating mass motor, an LRA produces vibration through the oscillatory motion of an internal mass near its resonant frequency, which enables faster response and more controlled haptic effects. This actuator has a diameter of 8 mm, a thickness of approximately 3.25 mm, a rated voltage of 1.8 V_{RMS}, an operating voltage range of 0.1 to 1.85 V_{RMS}, a resonant frequency of 235 Hz, a rated current of 80 mA maximum, a typical current of 58 mA, and a typical vibration force of 1.0 G under rated conditions. It is a Z-axis actuator and includes double-sided adhesive tape to facilitate mounting.

For more information check the product pages at these links: <https://www.vybronic.com/wp-content/uploads/datasheet-files/Vybronic-VG0832013D-datasheet.pdf>

5.3.1 CONNECTIONS

- 3.5 mm AUX cable → AudioAmp 2 Click line-in jack
- Tip (left channel) → Pixi Click analog output
- Ring (right channel) → Pixi Click analog output
- Sleeve (GND) → Pixi Click GND
- AudioAmp 2 Click (+) → LRA terminal 1 (red)
- AudioAmp 2 Click (−) → LRA terminal 2 (black)

5.3.2 PRACTICAL NOTES

1. Best performance is achieved when the actuator is driven close to its resonant frequency of 235 Hz.
2. Since this actuator includes adhesive tape for mounting, the mechanical fixation is simplified, but the mounting surface should still be flat and stable to obtain repeatable vibration performance.

5.4 NARROW BAND LINEAR RESONANT ACTUATOR

The Vybronic VL120628H is a linear resonant actuator (LRA) designed for haptic feedback applications. Unlike a conventional eccentric rotating mass motor, an LRA produces vibration through the oscillatory motion of an internal mass at its resonant frequency, which allows faster response and more precise haptic effects. This actuator has dimensions of $12 \times 6 \times 2.8$ mm, an X-axis vibration direction, a rated voltage of $2.0 V_{\text{RMS}}$, a resonant frequency of 200 ± 10 Hz, a rated current of $130 \text{ mA}_{\text{RMS}}$ maximum, and a typical acceleration of $1.8 \pm 0.5 G$ under rated conditions

For more information check the product pages at these links: <https://www.vybronic.com/wp-content/uploads/datasheet-files/Vybronic-VL120628H-datasheet.pdf>

5.4.1 CONNECTION

- 3.5 mm AUX cable → AudioAmp 2 Click line-in jack
- Tip (left channel) → Pixi Click analog output
- Ring (right channel) → Pixi Click analog output
- Sleeve (GND) → Pixi Click GND
- AudioAmp 2 Click (+) → LRA terminal 1 (red)
- AudioAmp 2 Click (−) → LRA terminal 2 (black)

5.4.2 PRACTICAL NOTES

1. Best performance is achieved when the actuator is driven close to its resonant frequency of 200 Hz.
2. A dedicated haptic driver is recommended for reliable operation and proper waveform generation.

3. The mounting condition affects the vibration response, so the actuator should be fixed carefully in the final mechanical structure.

5.5 THERMOELECTRIC PELTIER MODULE

The Same Sky CP40236 is a thermoelectric cooler (TEC), also known as a Peltier module. It generates a temperature difference between its two faces when a DC current is applied: one side becomes hot while the other becomes cold. Reversing the current direction swaps the hot and cold sides. The device is a single-stage module with dimensions of $20 \times 20 \times 3.6$ mm, wire leads, a maximum current of 4 A, a maximum voltage of 8.6 V, an internal resistance of approximately 2Ω , and a maximum temperature difference of about 66°C under specified operating conditions

For more information check the product pages at these links: <https://www.sameskydevices.com/product/resource/cp40.pdf>

5.5.1 CONNECTIONS

Since the CP40236 requires significantly more current than the Black Pill STM32F411 can provide directly, it must be driven through a motor driver and powered using an external supply. In this setup, an L298N motor driver board is used. The L298N is a dual H-bridge driver, so one bridge channel can be used to drive the Peltier module as a two-terminal load.

- Peltier terminal 1 (red) → L298N OUT1
- Peltier terminal 2 (black) → L298N OUT2
- External supply positive → L298N motor supply input
- External supply ground → L298N power ground
- Pixi Click MAX11300 GPO → L298N IN1
- Pixi Click MAX11300 GPO → L298N IN2
- L298N ENA → kept enabled with onboard jumper

The two controller signals connected to **IN1** and **IN2** define the polarity applied to the Peltier module. When the logic states are reversed, the direction of the current through the module is reversed, which swaps the hot and cold sides. In this configuration, the channel is continuously enabled through the ENA jumper, so no additional enable signal is required from the microcontroller.

5.5.2 PRACTICAL NOTES

1. A heat sink should be attached to the hot side of the Peltier module; otherwise, thermal performance decreases rapidly and overheating may occur.
2. For accurate thermal regulation, a temperature sensor such as a thermistor can be added and used in a closed-loop control system.
3. The Peltier module must not be powered directly from the Black Pill 3.3 V output or from USB.
4. The L298N board must be supplied from an external power source sized for the Peltier module.
5. The selected driver should be checked carefully, since Peltier modules can draw substantial current and the L298N also introduces voltage drop across the bridge.